

رهگیری و دنبال کردن هدف هوایی از پیش آموزش دیده شده توسط بینایی کامپیوتر و مدل YOLO آموزش دیده اختصاصی

تمامی تغییرات انجام شده در این فایل مارک خاکستری شده است

مقدمه:

در جنگ های امروزه با پیشرفت تکنولوژی نبرد از زمین به آسمان رفته و پدافند های پر هزینه دیگر به اندازه قبل کار آمد نیستند و بیشتر مانند اهداف ثابت هستند، از این رو استفاده از پدافند های ارزان قیمت ولی گسترده در سطح کشور آسمان کشور را امن تر می کند، و یکی از این روش ها روش های بینایی (اپتیکال) است، طرح من با استفاده از بینایی کامپیوتر و مدل YOLO آموزش دیده اختصاصی، تلاش دارد تا اهداف آموزش دیده را شناسایی و به هم به صورت فیزیکی و هم دیجیتالی دنبال کند و موقعیت آن را در دوربین حفظ کند و نقشی در امن تر کردن آسمان کشور ایفا کند.

هدف طرح:

هدف اصلی این طرح، طراحی و پیاده سازی یک سامانه ی بینایی محور برای شناسایی و رهگیری اهداف هوایی از پیش آموزش دیده است. سامانه با دریافت تصویر زنده از دوربین، ابتدا اهداف مورد نظر را توسط یک مدل اختصاصی YOLO شناسایی کرده و سپس با استفاده از الگوریتم رهگیری، موقعیت هدف را در فریم های متوالی حفظ می کند.

پس از شناسایی هدف، مختصات مرکز آن نسبت به مرکز تصویر محاسبه شده و سامانه بر اساس این اختلاف، دستور مناسب را برای تغییر زاویه ی افقی (Pan) و عمودی (Tilt) دوربین ارسال می کند. در نتیجه دوربین به صورت خودکار حرکت کرده و تلاش می کند هدف را در مرکز تصویر خود نگه دارد.

ساختار کلی سامانه:

سامانه از چند بخش اصلی تشکیل شده است:

1. دریافت تصویر از دوربین
 2. پردازش تصویر و تشخیص هدف
 3. رهگیری هدف در فریم‌های متوالی
 4. محاسبه موقعیت زاویه‌ای هدف
 5. کنترل مکانیزم Pan/Tilt
 6. کنترل میزان بزرگنمایی دوربین
 7. مدیریت وضعیت سامانه در زمان از دست رفتن هدف
- ارتباط میان بخش پردازش تصویر و سیستم کنترل دوربین از طریق پروتکل **UDP** انجام می‌شود. استفاده از UDP باعث می‌شود فرمان‌های کنترلی با تأخیر کم منتقل شوند و در صورت دریافت فرمان جدید، اطلاعات قدیمی باعث ایجاد صف و تأخیر در کنترل دوربین نشود.

دیتاست آموزش:

برای آموزش مدل نیاز به داده است و فرایند آماده سازی آن به شرح زیر است.

منبع داده و استخراج آن:

تمامی کدهای مربوط به این بخش (`extract_frames.py`, `auto_label.py`, `prepare_dataset.py`) در گیت هاب قرار دارند.

داده‌های آموزش این پروژه از یک ویدیوی ۸۰ ثانیه‌ای تهیه شده توسط دوربینی که از زمین کوادکوپتر را رصد می‌کرد جمع‌آوری شده است. ویدیو در محیط داخلی با نورپردازی طبیعی و پس‌زمینه‌های متنوع (دیوار، کفپوش، نورهای محیطی) ضبط شده است.

برای ساخت دیتاست، فریم‌ها با نرخ ۴ فریم بر ثانیه استخراج شدند. سپس فریم‌های تکراری یا بسیار مشابه با استفاده از معیار شباهت تصویری حذف شدند. در نتیجه این فرآیند، ۲۹۸ فریم یکتا برای برچسب‌زنی باقی ماند.

برچسب زنی:

برچسب گذاری در دو مرحله انجام شد.

1. برای برچسب زنی داده ها ابتدا از یک اسکریپت پایتون و مدل پایه (marie-kjelberg/drone-detector) استفاده

شد، که داده ها را با دقت حدودی ۸۵٪ برچسب گذاری کرد

2. سایر داده ها که به نادرست برچسب گذاری شده بودند، با استفاده از LabelImg به صورت دستی رفع اشکال و

برچسب گذاری شدند و همچنین تشخیص های نادرست (لامپ، طرح های دیوار و سطوح براق نیز) حذف شدند

تقسیم داده ها:

داده ها با نسبت ۷۰/۱۵/۱۵ تقسیم شدند.

کاربرد	تعداد داده	مجموعه
آموزش مدل	۱۹۶	Train
تنظیم پارامترها و انتخاب بهترین epoch در آموزش	۴۱	Val
ارزیابی نهایی	۴۳	Test

نکته مهم درباره روش تقسیم: از آنجا که تمام فریم ها از یک ویدیوی پیوسته استخراج شده اند، تقسیم تصادفی می تواند

باعث نشت داده (data leakage) شود؛ زیرا فریم های مجاور تقریباً یکسان هستند. به همین دلیل، تقسیم به

صورت متوالی (temporal) انجام شد تا فریم های هر بخش از نظر زمانی جدا باشند و ارزیابی واقعی تری حاصل شود.

مورد	مقدار
تعداد کل داده	۲۹۸
تعداد کلاس ها	۱ (کواد کوپتر)
ابزار برچسب زنی	LabelImg
روش تقسیم	Temporal
نسبت تقسیم	۷۰/۱۵/۱۵

آموزش مدل:

تمامی کد های مربوط به این بخش (`finetune_model.py,test_eval.py`) در گیت هاب قرار دارند.

مدل پایه استفاده شده در این پروژه، (`marie-kjelberg/drone-detector`) است، این مدل ، مدل تشخیص پهپاد عمومی است که روی مدل YOLO11n Fine Tune شده است، ما این مدل را روی دیتاست اختصاصی خودمان ، برای استفاده اختصاصی (Domain Specific) ، Fine Tune کردیم

پیکربندی آموزش مدل:

پارامتر	مقدار
مدل پایه	(marie-kjelberg/drone-detector)
تعداد پارامتر	~ 2.6 Million
Epochs	50(With Early Stopping)
Patience	15
Batch size	8
imgsz	640x640
Freeze Layers	10 First Layers
Optimizer	AdamW
Device	RTX4050 + CUDA 11.8

راهبرد برای Fine Tuning :

بدلیل کوچک بودن دیتاست، ۱۰ لایه اول شبکه را فریز کردیم تا دانش عمومی اش را از دست نده و فقط لایه های آخر (تشخیصی) را آموزش دادیم تا مدل برای هدف ما اختصاصی شود

نتیجه آموزش (بهترین مدل):

آموزش با Early Stopping در epoch 42 متوقف شد و بهترین نتیجه در epoch 27 به دست آمد. منحنی های آموزش (Loss and mAp) در تصویر زیر نمایش داده شده اند:

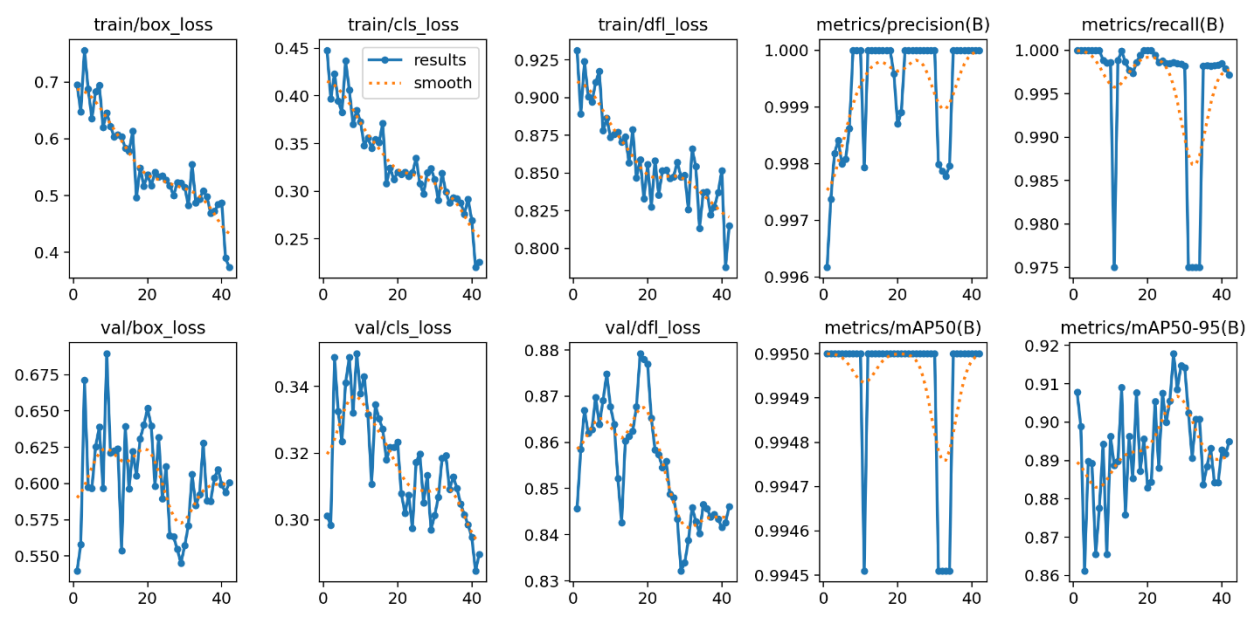


Figure 1 Training Results

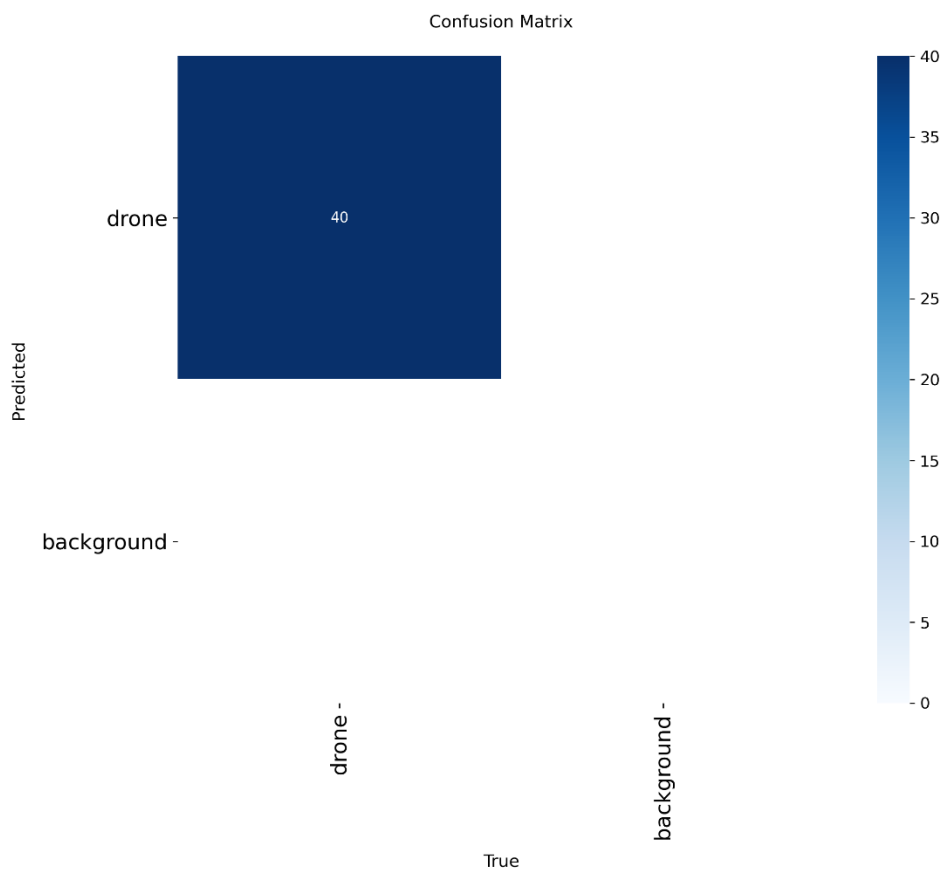


Figure2 Confusion matrix on Validation Data

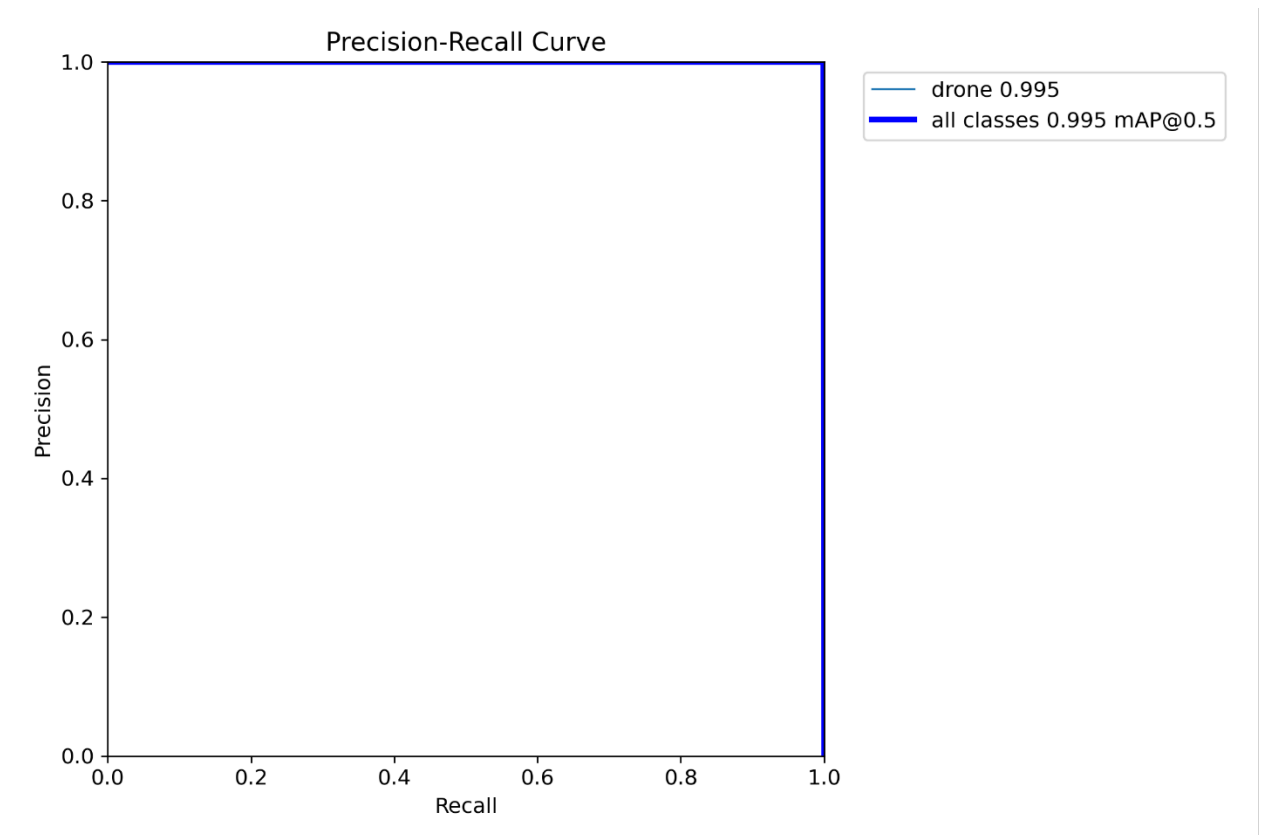


Figure 3 PR Curve on Validation Data

"لازم به ذکر است که مقادیر گزارش شده در این بخش مربوط به مجموعه *validation* است. ارزیابی نهایی روی مجموعه *test* در بخش «نتایج کمی» به صورت جداگانه ارائه شده است."

تشخیص هدف با مدل:

هسته‌ی بخش بینایی ماشین سامانه، مدل **YOLO** است. مدل مورد استفاده پس از آموزش اختصاصی، قادر است کلاس هدف مورد نظر را از میان تصویر دوربین تشخیص دهد.

در هر فریم، مدل مختصات حدودی جعبه‌ای که هدف را در بر می‌گیرد را استخراج می‌کند. از این محدوده، مرکز هدف به صورت زیر به دست می‌آید:

$$BCx = (Xmin + Xmax) / 2$$

$$BCy = (Ymin + Ymax) / 2$$

مرکز به دست آمده سپس برای محاسبه‌ی میزان انحراف هدف از مرکز تصویر استفاده می‌شود.

برای کاهش خطاهای مربوط به شناسایی‌های مستقل در فریم‌های متوالی، از الگوریتم **ByteTrack** به عنوان دنبال کننده استفاده شده است. این بخش کمک می‌کند هدف شناسایی شده در فریم‌های مختلف به عنوان یک هدف پیوسته دنبال شود و سیستم صرفاً به تشخیص مستقل هر فریم وابسته نباشد.

محاسبه زاویه حرکت دوربین:

صرفاً دانستن مختصات پیکسلی هدف برای کنترل دقیق دوربین کافی نیست؛ زیرا میزان جابه‌جایی پیکسلی هدف به میدان دید دوربین و نسبت تصویر نیز وابسته است.

در این طرح، مختصات هدف ابتدا از فضای پیکسلی به مختصات نرمال شده تبدیل می‌شوند. سپس با در نظر گرفتن میدان دید دوربین (FOV) و نسبت تصویر، یک بردار تقریبی سه بعدی از جهت مشاهده‌ی هدف ساخته می‌شود.

با استفاده از توابع مثلثاتی atan2 ، این بردار به زاویه‌های Pan و Tilt تبدیل می‌شود.

این روش نسبت به استفاده‌ی ساده از نسبت پیکسل‌ها، کنترل دقیق‌تری در نزدیکی لبه‌های تصویر ایجاد می‌کند و از ایجاد چرخش‌های غیرعادی دوربین جلوگیری می‌کند.

کنترل Pan و Tilt :

پس از محاسبه‌ی زاویه‌ی هدف نسبت به مرکز تصویر، مقدار حرکت دوربین محاسبه می‌شود.

فرمان تولیدشده دارای ساختاری مشابه زیر است:

$\text{PAN_TILT} \rightarrow \Delta\text{Pan} \Delta\text{Tilt}$

مقادیر محاسبه شده قبل از ارسال به دوربین محدود می‌شوند تا دوربین در یک فریم نتواند بیش از مقدار مشخصی حرکت کند.

همچنین برای جلوگیری از حرکات شدید و ناپایدار، یک ضریب کنترل برای میزان حرکت در نظر گرفته شده است. این ضریب باعث می‌شود دوربین به جای حرکت ناگهانی، به صورت تدریجی به سمت هدف حرکت کند.

در صورتی که هدف در قسمت‌های بالایی یا پایینی تصویر قرار گرفته باشد، میزان حرکت افقی نیز می‌تواند کاهش پیدا کند. این روش از چرخش‌های غیرضروری Pan در شرایطی که اصلاح Tilt اولویت بیشتری دارد جلوگیری می‌کند.

بزرگنمایی خودکار:

یکی دیگر از بخش‌های سامانه، کنترل خودکار Zoom است.

هنگامی که هدف شناسایی و قفل می‌شود، بزرگنمایی دوربین به صورت تدریجی افزایش پیدا می‌کند تا هدف بخش بزرگ‌تری از تصویر را اشغال کند. این افزایش به صورت ناگهانی انجام نمی‌شود، بلکه با یک ضریب شتاب نرم‌شده انجام می‌گیرد.

در صورت از دست رفتن هدف، بزرگنمایی نیز به صورت تدریجی به حالت اولیه بازمی‌گردد.

همچنین هنگام تغییر Zoom، میدان دید مؤثر دوربین تغییر می‌کند. بنابراین سامانه میدان دید جدید را مجدداً محاسبه کرده و از آن در محاسبات Pan و Tilt استفاده می‌کند. این موضوع برای جلوگیری از خطای زاویه‌ای پس از بزرگنمایی اهمیت زیادی دارد.

مدیریت وضعیت سامانه :

سامانه دارای چند وضعیت اصلی است:

Search یا جست‌وجو

در این وضعیت هدفی شناسایی نشده و دوربین در حالت جست‌وجو قرار دارد.

Locked یا قفل روی هدف

پس از شناسایی هدف، سامانه وارد حالت قفل می‌شود. در این حالت موقعیت هدف در هر فریم محاسبه شده و فرمان‌های Pan و Tilt برای دنبال کردن آن ارسال می‌شوند. هم‌زمان Zoom نیز می‌تواند افزایش پیدا کند.

Return یا بازگشت به حالت اولیه

اگر هدف برای تعداد مشخصی از فریم‌ها شناسایی نشود، سامانه در نظر می‌گیرد که هدف از میدان دید خارج شده است. در این حالت دوربین به صورت تدریجی به موقعیت اولیه خود بازمی‌گردد.

پس از نزدیک شدن دوربین به موقعیت اولیه، سامانه دوباره وارد وضعیت جست و جو می‌شود.

این ساختار باعث می‌شود سیستم در هنگام از دست دادن موقت هدف، بدون نیاز به دخالت کاربر بتواند فرآیند جست‌وجو را از سر بگیرد.

ارتباط میان نرم افزار و دوربین:

برای انتقال داده میان بخش پردازش تصویر و کنترل دوربین، چند کانال ارتباطی UDP در نظر گرفته شده است.

یک کانال برای ارسال فرمان های Pan/Tilt استفاده می شود و کانال دیگری وضعیت فعلی زاویه ی دوربین را به سیستم پردازش تصویر بازمی گرداند.

همچنین تصاویر دوربین به صورت فشرده شده دریافت شده و پس از Decode شدن توسط OpenCV، برای تشخیص و رهگیری در اختیار مدل YOLO قرار می گیرند.

در این معماری، سیستم بینایی ماشین نقش تصمیم گیرنده را دارد و سیستم کنترل دوربین وظیفه ی اجرای فرمان های حرکتی را بر عهده می گیرد.

فناوری ها و کتابخانه های مورد استفاده:

در پیاده سازی نمونه ی اولیه از فناوری های زیر استفاده شده است:

- **Python** به عنوان زبان اصلی پردازش و کنترل
- **OpenCV** برای دریافت، پردازش و نمایش تصاویر
- **NumPy** برای عملیات عددی و پردازش داده های تصویری
- **Ultralytics YOLO** برای تشخیص و رهگیری اهداف
- **Supervision** برای پردازش و نمایش محدوده هدف و اطلاعات تشخیص
- **UDP Socket** برای ارتباط سریع میان سامانه ی بینایی و کنترل دوربین
- **CUDA/GPU** برای اجرای سریع مدل یادگیری عمیق

اجرای مدل روی GPU باعث افزایش سرعت پردازش شده و امکان پردازش تعداد بیشتری از فریم ها در واحد زمان را فراهم می کند. اما اجرا روی CPU نیز ممکن ولی سرعت پردازش به شدت افت می کند

روند عملکرد سامانه:

روند کلی عملکرد سیستم به صورت زیر است:

دوربین ← دریافت تصویر ← پردازش تصویر ← YOLO ← تشخیص هدف → ByteTrack ← تعیین مرکز هدف ← محاسبه زاویه ← کنترل ← Pan/Tilt تغییر جهت دوربین ← دریافت فریم جدید

این چرخه به صورت پیوسته تکرار می شود.

در صورتی که هدف وجود داشته باشد، سیستم به حالت قفل وارد شده و دوربین خود را با هدف هماهنگ می کند. در صورت عدم مشاهده هدف برای تعداد مشخصی از فریم ها، سامانه وارد حالت بازگشت شده و پس از بازگشت به محدوده مرجع، مجدداً وارد حالت جست و جو می شود.

راهنمای اجرای سامانه در محیط شبیه سازی شده

سامانه از دو بخش مستقل تشکیل شده است: محیط شبیه سازی Unity و بخش پردازش Python. این دو بخش از طریق پروتکل UDP با یکدیگر ارتباط برقرار می کنند. (کد های یونیتی در فولدر Unity_Codes) در گیت اب قرار دارند.

پیش نیازها:

نرم افزار	نسخه
ویندوز	۱۰ یا ۱۱
پایتون	<3.10 >3.13
Pytorch	+2.0(Cuda)
Ultralytics	+ 8.3.0
Supervision	+0.24.0
Open CV	+4.9.0

همه کتابخانه های مورد نیاز در فایل requirements.txt موجود است.

برای استفاده از GPU از requirements-cuda.txt استفاده کنید

مراحل اجرا:

۱. استخراج محیط شبیه سازی و اجرای فایل EXE

فایل EXE را از فولدر RadarDroneDetection اجرا کنید

۲. اجرای بخش پردازش Python:

- فایل simulation_tracker.py را اجرا کنید:

- سامانه به صورت خودکار شروع به دریافت فریم از Unity می کند

- پس از شناسایی هدف، فرمان های Pan/Tilt ارسال می شوند

۳. تعامل با سامانه:

- کلید Q یا Ctrl + C: خروج از برنامه و ذخیره ی لاگ

- لاگ ها در پوشه logs/ ذخیره می شوند

- ویدیوی خروجی در پوشه logs/ ذخیره می شود

۴. تحلیل نتایج:

خود فایل پایتون یک فایل لاگ csv ایجاد میکند ، شما می توانید با CSV_PATH در plot_drawer.py نمودار خطا برحسب زمان را رسم کنید، نسبت زمانی آن کاملاً بر ویدیو تولید شده در log مطابق است

پارامترهای قابل تنظیم:

پارامتر	مقدار پیش فرض	توضیح
FOV	90	میدان دید دوربین
MAX_ROT_DELTA	8	حداکثر چرخش ارسالی در هر فریم
LOST_FRAME_THRESHOLD	20	تعداد فریم های مجاز برای از دست دادن هدف
PAN_AND_TILT_GAIN	0.5	ضریب کنترل حرکت

نتایج کمی:

مقایسه مدل پایه و Fine Tuned روی دیتای Test :

مدل	mAP@0.5	mAP@0.5:0.95	Precision	Recall
مدل پایه (marie-kjelberg/drone-detector)	0.918	0.693	0.960	0.809
مدل آموزش دیده اختصاصی (Fine Tuned)	0.990	0.794	0.976	0.970
بهبود	+ 7.8%	+ 14.7%	+1.7 %	+ 19.9%

تحلیل نتایج:

Fine Tuning ، بهبود قابل توجهی را دقت مدل در تمام معیار ها ایجاد کرده است.

1. Recall مدل ۱۹.۹ درصد افزایش یافته، یعنی که مدل اختصاصی حدود ۹۷٪ پهباد ها را تشخیص می دهد در حالی که مدل پایه حدود ۸۱٪ آنها را شناسایی می کند

mAP@0.5 از ۰,۹۱۸ به ۰,۹۹۰ افزایش یافت (بهبود نسبی ۷,۸٪). با توجه به کوچک بودن و یکسان بودن محیط مجموعه‌ی آزمون، این مقدار نباید به محیط‌های دیگر تعمیم داده شود.

2. mAP@0.5:0.95 مدل از ۰.۶۹۳ به ۰.۷۹۴ افزایش یافت. این معیار سختگیرانه‌تر، دقت جعبه‌های محدودکننده را می‌سنجد. بهبود ۱۴.۷٪ نشان می‌دهد مدل نه تنها پهباد را پیدا می‌کند، بلکه موقعیت آن را در این محیط اختصاصی دقیق‌تر تعیین می‌کند.

3. Precision از ۰.۹۶۰ به ۰.۹۷۶ رسید، یعنی نرخ هشدار نادرست (false positive) پایین ماند

تحلیل تاثیر رزولوشن بر مدل:

برای یافتن بهترین رزولوشن برای آموزش مدل و ورودی آن سه مدل به رزولوشن های متفاوت آموزش دید و با ۲ یا ۳ رزولوشن متفاوت هر کدام آزمایش شدند

Precision	Recall	mAP@0.5:0.95	mAP@0.5	Eval imgsز	Train imgsز
0.9229	0.8553	0.5858	0.9164	320	320
0.9957	1.0000	0.7313	0.9950	640	320
0.9472	0.4272	0.3807	0.5156	320	640
0.9761	0.9708	0.7945	0.9900	640	640
1.0000	0.9972	0.7458	0.9950	960	640
0.9902	0.9286	0.7261	0.9884	640	960
0.8365	0.6094	0.5376	0.7615	960	960

رزولوشن ۶۴۰ نقطه‌ی بهینه‌ی آموزش است و ارزیابی در محدوده‌ی ۶۴۰ تا ۹۶۰ بهترین عملکرد را می‌دهد. ترکیب «آموزش ۶۴۰ / ارزیابی ۹۶۰» بالاترین قابلیت اطمینان را فراهم می‌کند (Precision = 1 , Recall = 0.97)، در حالی که «آموزش ۶۴۰ / ارزیابی ۶۴۰» بهترین دقت جعبه را دارد (mAP@0.5:0.95 = 0.795). کاهش رزولوشن ارزیابی به ۳۲۰ باعث افت شدید عملکرد می‌شود (mAP@0.5:0.95 = 0.381) ، که اهمیت تطابق رزولوشن آموزش و ارزیابی را نشان می‌دهد.

اما با ارزیابی میدانی روی ویدیو به شکل زنده ، مشخص شد که مدل ۶۴۰ با ورودی ۶۴۰ بهترین تعادل را دارد، و برخی مقادیر به دلیل کوچک بودن دیتاست مقداری نویز دارند

(برای ارزیابی عملکرد مدل در ویدیو واقعی ضبط شده، می تونید از فایل `video_interfere.py` یا ویدیو اجرا شده آن(`finetuned_test.mp4`) استفاده کنید)

ارزیابی سناریو محور(محیط شبیه سازی شده):

برای ارزیابی عملکرد سامانه در یک سناریو واقعی ۴ تست در نظر می گیریم

(در این آزمایش از مدل پایه استفاده شده است و هدف اصلی آن آزمایش Pan Tilt است)

مکان	وضعیت
نزدیک مرکز تصویر	ثابت
نزدیک مرکز تصویر	در حال حرکت
نزدیک لبه تصویر	ثابت
نزدیک لبه تصویر	در حال حرکت

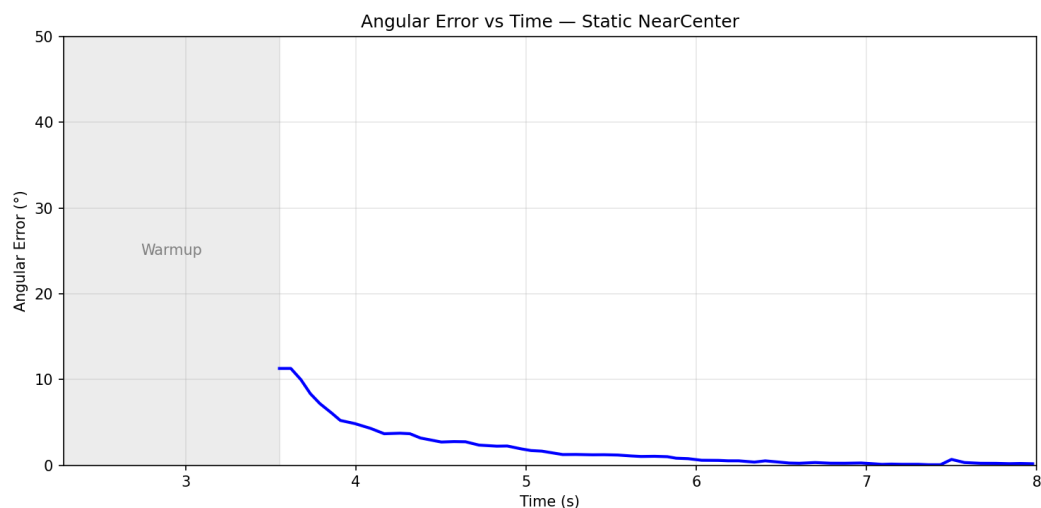


Figure 4: خطای زاویه‌ای نسبت به زمان — سناریوی هدف ساکن نزدیک مرکز تصویر

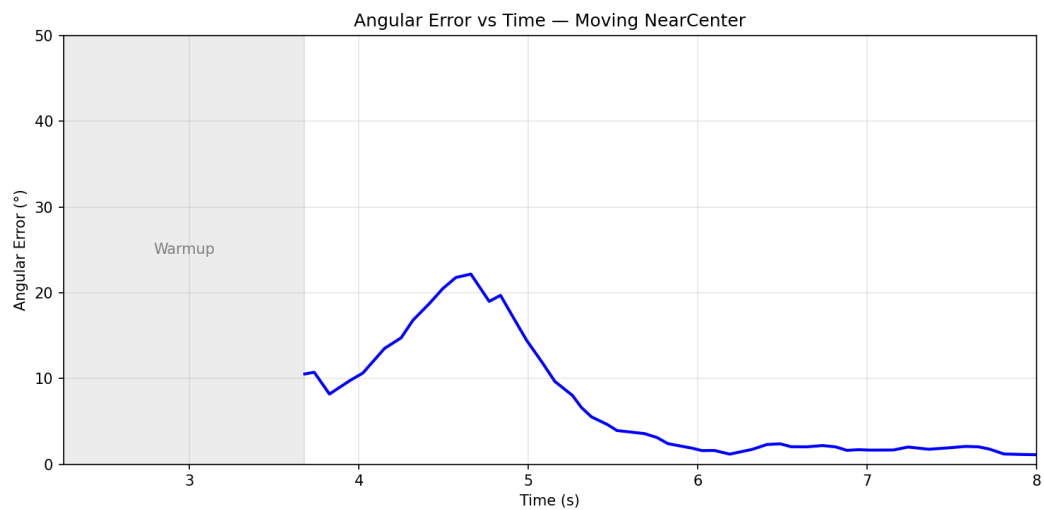


Figure 5: خطای زاویه‌ای نسبت به زمان — سناریوی هدف متحرک نزدیک مرکز تصویر

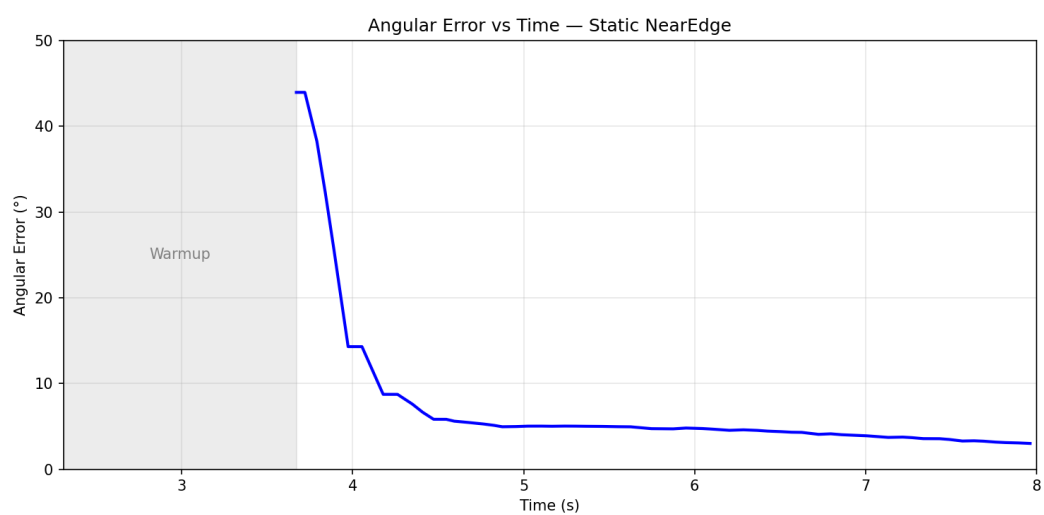


Figure 6: خطای زاویه‌ای نسبت به زمان — سناریوی هدف ثابت نزدیک لبه تصویر

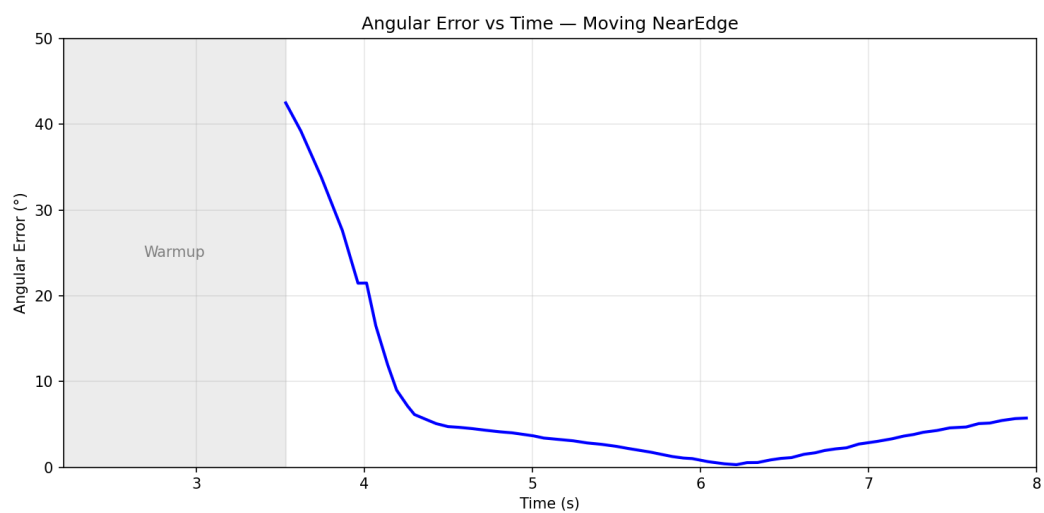


Figure 7: خطای زاویه‌ای نسبت به زمان — سناریوی هدف متحرک نزدیک لبه تصویر

در سناریوهای ساکن (مرکز و لبه)، سامانه عملکرد پایدار نشان داد. در سناریوی «هدف ساکن در مرکز»، خطای اولیه حدود ۱۱ درجه بود که در حدود ۲ ثانیه به زیر ۲ درجه کاهش یافت. در سناریوی «هدف ساکن در لبه»، خطای اولیه به حدود ۴۴ درجه می‌رسید که سامانه در کمتر از ۱.۵ ثانیه آن را به زیر ۵ درجه رساند.

در سناریوهای متحرک، نرخ رهگیری ۹۹٪ و میانگین خطای زاویه‌ای ۲.۵ درجه بود. در سناریوی «هدف متحرک در مرکز»، خطا پس از قفل شدن ابتدا تا حدود ۲۲ درجه افزایش یافت (ناشی از تقدم pan بر tilt) و سپس به سرعت به زیر ۵ درجه بازگشت. این رفتار به دلیل طراحی کنترلر است که هنگام نزدیک شدن هدف به بالای تصویر، حساسیت pan را کاهش می‌دهد تا از چرخش‌های ناخواسته جلوگیری شود.

سناریوی «هدف متحرک در لبه» چالش‌برانگیزترین حالت بود، اما سامانه موفق شد در ۹۸٪ از فریم‌ها هدف را حفظ کند و خطا را به زیر ۵ درجه برساند.

در تمام چهار سناریو، سامانه توانست هدف را در کمتر از ۲ ثانیه به مرکز تصویر برساند و خطای زاویه‌ای پایدار زیر ۵ درجه حفظ کند. این عملکرد نشان‌دهنده‌ی کارایی بالایی حلقه‌ی کنترل Pan/Tilt و پایداری الگوریتم رهگیری است.

سرعت مدل و سامانه:

مدل آموزش دیده و پایه در آزمایش‌ها تاخیری حدود ۴.۹ میلی ثانیه را ثبت کردند که حتی بالاتر از سرعت ضبط فریم بیشتر دوربین هاست، پردازش سریع تر از دریافت فریم انجام می شود که بار موجود بر GPU (کارت گرافیک) را کاهش می دهد.

در محیط شبیه سازی شده، به دلیل ارسال فریم ها از برنامه توسط کانال های UDP و پروسه های تبدیل فریم به JPG، ارسال آن به پایتون ، بازگشایی آن در پایتون به فریم خام ،انجام شناسایی و رهگیری کل حلقه در حدود 15 FPS اجرا می شود این محدودیت در محیط شبیه سازی است و در استقرار فیزیکی با اتصال مستقیم دوربین USB وجود ندارد.

نوآوری و مزایای طرح:

مهم‌ترین ویژگی این طرح، ترکیب چند فناوری در یک سامانه‌ی یکپارچه است. برخلاف یک سیستم تشخیص تصویر ساده که تنها وجود هدف را اعلام می‌کند، این سامانه نتیجه‌ی تشخیص را مستقیماً وارد حلقه‌ی کنترل دوربین می‌کند. به این ترتیب، سیستم از یک ساختار صرفاً تشخیصی به یک ساختار تشخیص، رهگیری و کنترل تبدیل می‌شود.

از دیگر مزایای این روش می‌توان به موارد زیر اشاره کرد:

- امکان آموزش مدل برای اهداف مشخص و مورد نظر
- قابلیت پردازش بلادرنگ با استفاده از GPU
- کاهش وابستگی به اپراتور انسانی
- قابلیت استفاده از دوربین‌های متحرک PTZ
- کنترل خودکار Zoom
- مدیریت از دست رفتن هدف
- ارتباط کم‌تأخیر میان بخش‌های مختلف
- قابلیت توسعه برای استفاده از چند دوربین
- امکان توسعه Tracker و الگوریتم کنترل در آینده

محدودیت‌ها:

- دیتاست واقعی از یک ویدیوی ۸۰ ثانیه‌ای داخلی با یک پهپاد و پس‌زمینه‌های محدود تهیه شده و مجموعه‌ی آزمون از همان ویدیو است؛ بنابراین عملکرد در فضای باز، فواصل زیاد و برابر پس‌زمینه‌های متنوع هنوز ارزیابی نشده است.
- نمونه‌ی منفی (پرنده، هواپیما، بادبادک و اشیای مشابه) در داده‌ها وجود ندارد یا بسیار کم است؛ نرخ هشدار نادرست در میدان بررسی نشده است.

- اندازه‌ی مجموعه‌های آزمون کوچک است و آزمایش‌ها یک‌بار اجرا شده‌اند؛ تفاوت‌های کوچک بین نتایج ممکن است ناشی از نویز باشد. آزمایش رزولوشن روی مجموعه‌ی آزمون انجام شده و تحلیل حساسیت است، نه روش انتخاب مدل.
- ارزیابی حلقه‌ی کنترل در محیط شبیه‌ساز انجام شد؛ آزمون سخت‌افزاری با هدف انسانی و به‌صورت ضبط ویدیو انجام شده.
- بزرگ‌نمایی دیجیتال است و رزولوشن مؤثر اهداف کوچک را کاهش می‌دهد.
- سامانه در حال حاضر فقط یک هدف را دنبال می‌کند، کنترل آن تناسبی ساده بدون پیش‌بینی حرکت است.
- این طرح نمونه‌ی اولیه‌ی تشخیص و رهگیری است و قابلیت خنثی‌سازی یا ارزیابی تهدید ندارد.

تست فیزیکی:

به دلیل نبود دسترسی به کوادکوپتر در زمان بازبینی تست فیزیکی روی انسان با مدل YOLO26m انجام شد که فیلم مربوطه با نام `physical_test.mp4` در ریپازیتوری گیت هاب قرار دارد

نتیجه‌گیری:

این طرح یک نمونه‌ی اولیه از یک سامانه‌ی بینایی‌محور برای تشخیص و رهگیری خودکار اهداف هوایی ارائه می‌دهد. در این سامانه، مدل YOLO وظیفه‌ی شناسایی هدف، ByteTrack وظیفه‌ی حفظ هدف در طول حرکت و الگوریتم کنترل وظیفه‌ی تبدیل موقعیت تصویری هدف به فرمان حرکتی دوربین را بر عهده دارد.

ترکیب این اجزا امکان ایجاد سامانه‌ای را فراهم می‌کند که پس از مشاهده‌ی هدف، بدون نیاز به کنترل مداوم اپراتور، دوربین را به سمت آن هدایت کرده و تلاش می‌کند هدف را در میدان دید حفظ کند.

در مراحل بعدی می‌توان سامانه را با استفاده از مدل‌های دقیق‌تر، آموزش روی داده‌های متنوع‌تر، بهبود الگوریتم پیش‌بینی حرکت هدف، کالیبر کردن دقیق دوربین، استفاده از چند دوربین و روش‌های پیشرفته‌تر رهگیری توسعه داد.

در نهایت، هدف پژوهشی این پروژه بررسی قابلیت‌های بینایی ماشین و یادگیری عمیق در سامانه‌های خودکار تشخیص و رهگیری اهداف و ارزیابی عملکرد آن‌ها در شرایط مختلف تصویری و حرکتی است.

سورس کد، فیلم عملکرد، کتابخانه‌های لازم و محیط شبیه‌سازی در گیت هاب موجود است

<https://github.com/mohammadJavad12/Aerial-Drone-Detection-YOLO-PTZ>

منابع:

Quadcopter Controller

<https://github.com/simeonradivoev/Quadcopter-Controller>

Unity Standard Assets

https://assetstore.unity.com/?q=standard%20assets&srltid=AU7gw4XvXePY-g0ZNMpldjDvDAfM_5W248klyJp4HhHz6zr76zDESZn

Ultralytics

<https://docs.ultralytics.com>

Base Model

<https://huggingface.co/marie-kjelberg/drone-detector>

ByteTrack

<https://github.com/ifzhang/ByteTrack>

supervision

<https://github.com/roboflow/supervision>

Labellmg

<https://github.com/HumanSignal/labellmg>

Zoom Part

<https://github.com/harimkang/openCV-with-Zoom>

OpenCV

<https://opencv.org>

NumPy

<https://numpy.org>

PySerial

<https://github.com/pyserial/pyserial>

سهم نویسنده:

بخش پردازش و رهگیری: (Python)

- جمع‌آوری و برچسب‌زنی دیتاست اختصاصی (298 داده)
- Fine-tuning مدل پایه روی دیتاست محلی
- ارزیابی کمی مدل (mAP, Precision, Recall)
- تحلیل رزولوشن (7 ترکیب آموزش/اجرا)
- پیاده‌سازی منطق تشخیص و رهگیری هدف
- محاسبات Pan/Tilt و کنترل سروو
- ماشین حالت (Search/Locked/Return)
- ارتباط UDP و Serial
- فرمول‌های تبدیل پیکسل به زاویه

بخش شبیه‌ساز: (Unity C#)

- کدهای Unity با استفاده از پروژه‌های متن‌باز و ابزارهای تولید کد نوشته شده‌اند
 - کنترلر کوادکوپتر simeonradivoev/Quadcopter-Controller :
 - Unity Standard Assets
 - ویرایش و تنظیم برای سازگاری با سامانه‌ی Python
-